Will It Fall? Diffusion and Flow-Matching World Models Underestimate the Risk of Failure

Anonymous Authors

Abstract—World models let a robot imagine what might happen before it acts. Many recent world models are diffusion or flow-matching models that sample possible futures. For safety, one of the most useful things such a model can tell us is how likely a failure is. We test this directly. We simulate a Unitree Go2 quadruped walking on varied terrain while random pushes hit it. At 1,149 test states we fork the simulator 128 times, so we know the true probability that the robot falls in the next two seconds. We find two problems. First, a common logging choice, stopping the log shortly after a fall, hides almost all early falls from the model and halves the predicted risk. Second, even with clean data and 64 sampling steps, diffusion and flow-matching world models predict only 57% and 65% of the true risk at uncertain states. More sampling steps close this gap only slowly: with 512 steps they still predict 77% and 83%. A classifier trained on the same data with the same network is nearly unbiased (92%). A coarse-to-fine model that first samples a rough version of the future and then adds detail, a continuous form of next-concept modeling (C-NCM), removes the bias (105%, 95% CI 95–117%), and with only 96 network passes it already matches what flow matching reaches with 512. The gain is in overall calibration, not in ranking: all models, including the classifier, rank states equally well, and simple recalibration on held-out data closes the gap. The main test was pre-registered.

I. Introduction

A robot that plans with a learned world model needs to know when a plan could end badly. Think of a quadruped climbing stairs that is about to be pushed. It may catch itself, or it may fall. A useful world model should not only produce futures that look right. It should also produce falls about as often as they really happen. If it shows falls too rarely, a planner that trusts it will take risks it does not see.

Diffusion [1], [2] and flow matching [3], [4] are now common choices for world models and trajectory models [5]–[9]. They are usually judged by how realistic their samples look or by the error of a single predicted future. Whether they get probabilities right is rarely checked, and for a good reason: in the real world, and in almost every dataset, each state happens once and only one future is ever seen. You cannot tell whether a state had a 10% or a 60% chance of failure from one outcome.

Simulation removes this limit. We can copy the full state of the physics engine and the controller, then run many futures from that exact state that differ only in the random pushes that will arrive. The fraction of these futures that end in a fall is the true fall probability of that state. We call this branching ground truth. We then ask a world model for 128 sampled futures from the same state and count how many fall.

With this test we found the following.

- A data pitfall. Our first dataset stopped logging one second after each fall, a common choice. Because every training window needs two seconds of future, windows that start just before a fall were silently dropped. Only 4% of training windows with a fall showed it in the first second, while 51% of real falls happen there (Fig. 3). This alone halved the predicted risk.
- A model bias. With clean data and 64 sampling steps, diffusion and conditional flow matching (CFM) still predict only 57% and 65% of the true risk at uncertain states. Eight times more steps (512) helps but does not close the gap (77% and 83%). A classifier trained on the same data, with the same inputs and network, predicts 92%. So the bias comes from the generative models, not from the data.
- A fix. C-NCM, a continuous version of next-concept modeling [10], samples a 4-step summary of the future first and then adds detail in five more levels. It predicts 105% of the true risk (95% CI 95–117%). At equal bias it needs far less compute: 96 network passes give the 83% that CFM needs 512 passes for.
- Clear limits. C-NCM does not rank states better than the other models. Every model, including the classifier, reaches about the same per-state accuracy (AUC near 0.90), and isotonic recalibration of CFM on held-out data matches C-NCM. Most states that are certain to fall, and that all models miss, are deterministic failures of the controller rather than effects of random pushes.

We wrote down our hypotheses before running the final test (pre-registration), including the cases where our method would lose. Two of four hypotheses held. We report all four.

II. Related Work

World models for robots. Learned dynamics models have long been used for planning and policy learning [11]–[14], including on real robots [15] and legged robots [16]. More recent models generate whole trajectories or videos with diffusion: Diffuser plans by denoising trajectories [5], DIAMOND and GameNGen simulate games with diffusion [6], [9], and DWM and PolyGRAD predict multi-step futures for reinforcement learning [7], [8]. Diffusion forcing mixes

next-step and full-sequence denoising [17]. These works mostly measure sample quality or downstream return, not whether the probability of rare outcomes is right.

Evaluating world models. Yu et al. argue that world models for embodied decision-making should be judged by how well they support decisions, with uncertainty calibration as one of the key axes [18]. dWorldEval estimates policy success rates with a world model and compares them with real execution [19]. Multi-future ground truth exists in other fields: the Garden of Forking Paths gives several plausible futures for pedestrians [20], ForecastBench-Sim re-seeds a strategy game to score forecasts [21], and Twin Rollouts forks a simulator to create counterfactual targets for video world models [22]. Our branches are exact copies of the physics and controller state, and the quantity we score is the probability of failure.

Diffusion, flow matching and rare modes. Diffusion with a cosine schedule [23], DDIM sampling [24] and flow matching with a linear path [3], [4], [25], [26] belong to the same family of Gaussian-path models [27], and in our results they behave alike. Several works report that such models give rare modes too little mass. SubFlow shows that class-conditional flows trained with a squared loss learn a frequency-weighted average of sub-modes and neglect rare ones [28]. Regional probability matching targets wrong mode frequencies in flow matching [29]. Other work steers diffusion toward low-density or minority samples [30], [31], and a diffusion model of windstorms underestimated the most intense events [32]. Mode interpolation can also create samples between modes [33]. In a world model the rare mode is often the failure, which is exactly the outcome a planner must not miss.

Coarse-to-fine generation. Next-scale prediction generates images from coarse to fine resolution [34], and cascaded diffusion stacks models at growing resolution [35]. Next-concept modeling (NCM) encodes text into a hierarchy of discrete residual codes and generates one level at a time, all codes of a level in parallel [10]; its tokenizer uses nested dropout [36]. We keep NCM's levels, residuals, attention pattern and corruption, but make every level continuous and generate it with flow matching.

Calibration and failure prediction. We score probabilities with the Brier score [37], a proper scoring rule [38]. Neural networks are often miscalibrated [39], and isotonic regression is a standard post-hoc fix [40], [41]. Weather forecasting checks the calibration of diffusion ensembles against observations [42]. In robotics, falls have been predicted with classifiers [43], and safety critics learn the chance of future failure [44], [45]. Recent work models disturbances inside world models for safe planning [46] or detects failures with a flow-matching world model [47]. Counterfactual reasoning with learned models goes back to structural causal models in policy search [48].

Episode ends and censoring. How an episode ends matters in reinforcement learning [49], and many datasets end an episode when the agent falls [50]. Survival analysis has long dealt with outcomes that are cut off before they are observed [51]. The pitfall in Section V is a selection effect of the same kind, caused by building fixed-length training windows from episodes that end soon after a failure.

III. Setup

A. Robot, controller and terrain

We use the Unitree Go2 model from MuJoCo Menagerie [52] in MuJoCo [53]. A model-based controller runs at 50 Hz: a gait scheduler, a stance-force balancer that spreads the body wrench over the stance feet by least squares with friction-cone clipping, Raibert-style foot placement [54] and analytic leg inverse kinematics. Joint PD control with feed-forward torque runs at 500 Hz, as on the real robot. Each episode lasts 10 s and has a random terrain type (flat, rough, slope or stairs), friction (0.4–1.2), payload (0–3 kg), motor strength ($\pm 10\%$), joint damping ($\times 0.7$–1.3) and heading. Commands change in random segments that pick a gait (stand, walk, trot, bound, pace) and a velocity.

B. Random pushes

Each episode has a push level: none, light, medium or strong. After the first second, pushes arrive at random times (a Poisson process, 0.4 per second). A push changes the body's horizontal velocity by up to 0.6, 1.2 or 1.8 m/s in a random direction and adds a small random yaw rate. Pushes use their own random number stream, separate from commands, so we can change the pushes without changing anything else.

C. Prediction task

The world model gets the current state (46 numbers: base height, gravity direction in the body frame, body linear and angular velocity, 12 joint angles, 12 joint velocities and the four foot positions in the body frame), the last 0.5 s of states, a 7×7 height scan around the robot, a 17×17 terrain height map (± 2 m), the episode parameters and the planned commands for the next 2 s. It must predict the next 100 states (2 s). The model knows the push level of the episode but not when pushes will come. Those future pushes are the randomness the model has to represent. A future counts as a fall if the body tilts more than 1 rad from vertical or the base drops below 0.12 m at any step.

D. Branching ground truth

At an anchor state we copy the MuJoCo state and the full controller state (including gait phase and internal memory), keep the command random stream, and give each copy a new push seed. Copies with the same push seed reproduce each other exactly. We run 128 copies for 2 s and take the fraction that fall as the true fall probability p.

The test set has 320 new episodes that were never used before the final run, with four random anchor times each

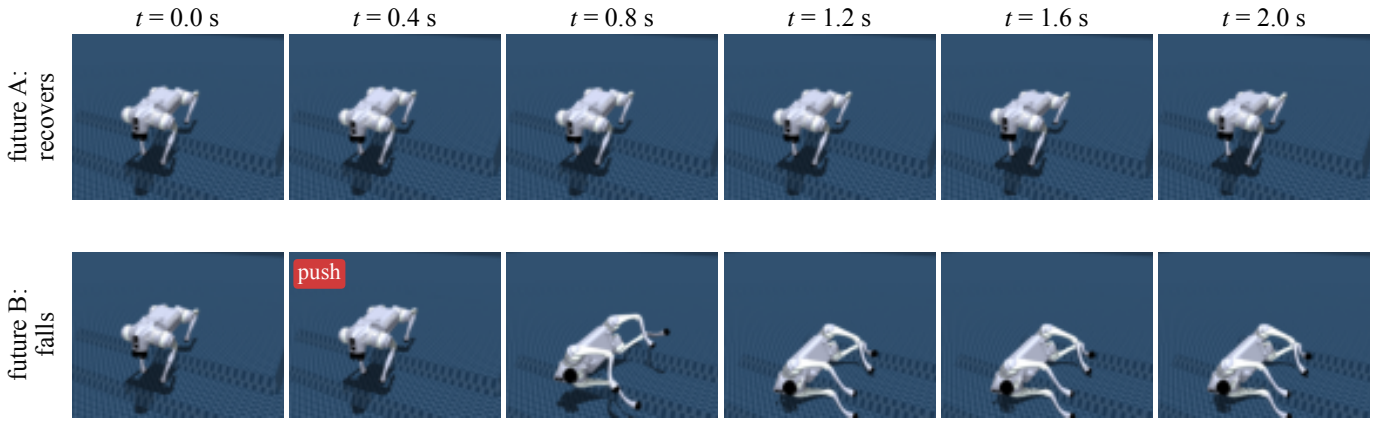


Fig. 1. One state, two futures. The robot is simulated up to a moment on stairs, then the simulator and controller are copied exactly. Both copies receive the same commands. They differ only in the random pushes that arrive later. In future A the robot keeps walking; in future B a push in the first 0.4 s makes it fall. Repeating this 128 times gives the true fall probability of the starting state (here 34%). A world model is asked for 128 futures from the same state, and we compare how many of them fall.

between 1.5 s and 7.6 s (anchors after a fall are skipped), giving 1,149 anchors. We call an anchor a fork when $0.05 < p < 0.95$, meaning the outcome really depends on the pushes; there are 319 forks, with mean true fall probability 0.164. Anchors in push-free episodes are deterministic (p is exactly 0 or 1). A separate set of 1,163 anchors from another 320 episodes is used only to fit recalibration maps.

E. Training data

We generate 6,016 episodes and split them by seed into 4,812 training, 602 validation and 602 test episodes. Training windows start at every step where the robot has not fallen yet and include the next 100 states, which gives about 1.78 million windows; 6.4% of them contain a fall.

F. Metrics

For each model and anchor we draw 128 futures and take the fraction that fall as the predicted probability $\hat{p}$. Our main measure is the risk ratio at forks: the mean of $\hat{p}$ over forks divided by the mean of p over forks. A value of 1 means no overall bias; below 1 means the model underestimates risk. We also report the Brier score $\frac{1}{n}\sum(\hat{p}-p)^2$ on forks and on all anchors, the area under the ROC curve (AUC) with each anchor weighted by p as a fall and $1-p$ as a non-fall, and the energy distance [55] between sampled and true futures. Predictions are averaged over three training seeds. Confidence intervals are 95% bootstrap intervals over anchors with 10,000 resamples [56]; differences between models are paired on the same anchors.

IV. Models

All world models share one transformer backbone [57] (width 256, 4 layers, 4 heads). The inputs become a prefix of tokens and the future becomes 100 tokens, one per step, each with the planned command of that step. Everything else, including the training data, steps and optimizer, is the same for all models: 40,000 steps, batch 256, AdamW [58] with learning rate 3×10^{-4}, a one-cycle schedule, weight decay 0.01 and gradient clipping at 1. We train three seeds of each model.

Diffusion. Variance-preserving diffusion with a cosine schedule [23]; the network predicts the clean future, and we sample with 64 DDIM steps [24].

CFM. Flow matching with a straight path between noise and data [3], [4]; the network predicts the clean future, and we sample with 64 Euler steps.

C-NCM. The future x (100 steps) is split into six levels with lengths $\ell = (4, 8, 16, 32, 64, 100)$. Let D_ℓ average-pool a trajectory in time to ℓ steps and U linearly interpolate it back to 100 steps. The levels are residuals:

$$r_1 = D_{\ell_1}(x), \qquad r_k = D_{\ell_k}\Big(x - \sum_{j<k} U(r_j)\Big), \qquad (1)$$

so $\sum_k U(r_k) = x$. Level k is a flow-matching model over its ℓ_k tokens, conditioned on the inputs and on all coarser levels, with block-wise attention and a shared prefix as in NCM [10]. Fig. 2 shows how the levels build up a trajectory. The first level is a 4-step summary of the next two seconds, so whether the robot ends up on the ground is already visible there. As in NCM, during training some tokens of the coarser levels are replaced by noise (rates falling linearly from 0.75 to 0.25 across levels), so that each level learns to tolerate mistakes made by the levels before it. The loss of level k is weighted by $\ell_k^{1-\alpha}$ with $\alpha = 0.5$. At test time we sample level by level with 64 Euler steps per level. Unlike NCM, there is no vector quantization and no categorical distribution; every level is continuous. With 64 steps per level, C-NCM uses $6\times64 = 384$ network passes per sample against 64 for CFM; on our GPU (RTX A5000) drawing 128 futures takes 1.9–2.0 s per anchor with C-NCM and 0.73 s with CFM. Section VI-F compares the models at equal numbers of passes.

Classifier. To see whether the inputs hold the information needed to predict a fall, we train a plain classifier

with the same encoders and transformer. A class token reads the prefix and the planned commands, and the model is trained with log-loss on the fall label of each training window. Because the terrain map and parameters identify each training episode, this classifier memorizes training episodes quickly, so we keep the checkpoint with the lowest log-loss on validation episodes (after 3,500 steps; validation log-loss 0.188, versus 0.259 for a constant prediction).

V. A Data Pitfall: Logging Stops After a Fall

Our first dataset stopped logging 50 steps (1 s) after a fall, which saves space and is a common choice. Each training window, however, needs 100 future steps. A window that starts less than 50 steps before a fall would run past the end of the log, so it is never created. As a result, only 4% of the training windows that contain a fall show the fall in the first second of the window, while 51% of the real falls in our test forks happen in the first second (Fig. 3). The models learn that a fall rarely happens soon, and they predict far too little risk: CFM and diffusion trained on this data predict 36% and 31% of the true risk at forks, about half of what the same models predict after the fix (Fig. 4).

We fixed the generator to log 100 steps after a fall and regenerated all 6,016 training episodes from the same seeds. All states before each fall are identical to the old data (largest difference 0), so the only change is the longer tail. Our data loader now refuses data whose tail after a fall is shorter than the model horizon. All results below use the fixed data unless we say otherwise.

The lesson is general. Any dataset that ends episodes at a failure, or soon after it, under-represents early failures in fixed-length windows unless the windows are built with care, for example by padding with the post-failure state. Our earlier pre-registered test on the truncated data (Appendix B) passed all its hypotheses, including a per-state accuracy gain for C-NCM; that accuracy gain did not survive the fix. We report it to show how easily such a pitfall can produce convincing but wrong results.

VI. Results

A. Pre-registered test

Before the final run we wrote down four hypotheses and the analysis plan (Appendix A):

- H1 CFM and diffusion underestimate risk at forks even with clean data (upper end of the ratio CI below 0.8).
- H2 C-NCM is less biased than CFM (CI of the ratio difference above 0).
- H3 C-NCM has a lower fork Brier score than CFM.
- H4 C-NCM has a lower fork Brier score than CFM after isotonic recalibration.

H1 and H2 hold; H3 and H4 do not. Table I and Fig. 4 give the numbers. CFM predicts 65% of the true risk at forks (95% CI 57–74%) and diffusion 57% (50–65%). C-NCM predicts 105% (95–117%); its ratio is 0.40 higher than CFM's (CI 0.34–0.47). Fig. 5 shows one fork: 44 of 128 true futures fall, CFM samples 18 falls, diffusion 12 and C-NCM 46.

B. Calibration across risk levels

Fig. 6 breaks the comparison down by true risk. For anchors with true probability between 0.05 and 0.5, CFM and diffusion lie below the diagonal; at true risk 0.05–0.2 (265 anchors, mean 0.108) they predict 0.074 and 0.064, while C-NCM predicts 0.128 and the classifier 0.111. C-NCM over-predicts slightly at safe anchors ($p < 0.05$: 0.015 against a true 0.006). All models fall far short at anchors with $p > 0.5$, which we look at in Section VI-E.

C. Per-state accuracy and recalibration

Getting the average right is not the same as getting each state right. On per-state accuracy C-NCM does not win. Its fork Brier score is 0.0274 against 0.0246 for CFM; the difference, +0.0028, has a CI from −0.0029 to +0.0091, so we cannot tell them apart. All three generative models rank anchors equally well (AUC 0.894–0.902). Its energy distance is worse (0.431 against 0.373), so its sampled trajectories are less accurate in detail, even though they fall at the right rate.

Because the models rank states equally well, the CFM bias can be removed after training. An isotonic map from predicted to true probability, fit on the separate calibration set, brings CFM's fork Brier score to 0.0245, the same as recalibrated C-NCM (difference +0.0003, CI −0.0048 to +0.0053). On all anchors, raw C-NCM is slightly worse than recalibrated CFM (+0.0033, CI 0.0007 to 0.0061). Recalibration needs held-out data with known outcomes and fixes the predicted number, not the sampled futures; C-NCM gets the rate right in its samples without extra data. We return to what this means in Section VII.

D. Is the information in the inputs?

One explanation for the shared accuracy ceiling would be that generative training throws away information about falls that the inputs contain. The classifier tests this. It sees exactly the same inputs and has the same network, but is trained only to predict the fall. It does not do better: fork Brier 0.0271, AUC 0.881. So, with this data and network size, the inputs and the amount of data seem to set the limit, not the way the world model is trained. The classifier is, however, close to unbiased overall (ratio 0.92, CI 0.82–1.04). This is the key control for our main claim: a model trained on the same data does not underestimate risk the way diffusion and CFM do, so their bias is not a property of the data.

E. States that are certain to fall

Forty-one anchors fall in at least 95% of their futures. All models give many of them a low probability: CFM predicts below 0.4 for 19 of them, diffusion for 22, C-NCM for 18 and the classifier for 17. Fig. 7 shows why these are hard. Nineteen of the 41 come from push-free

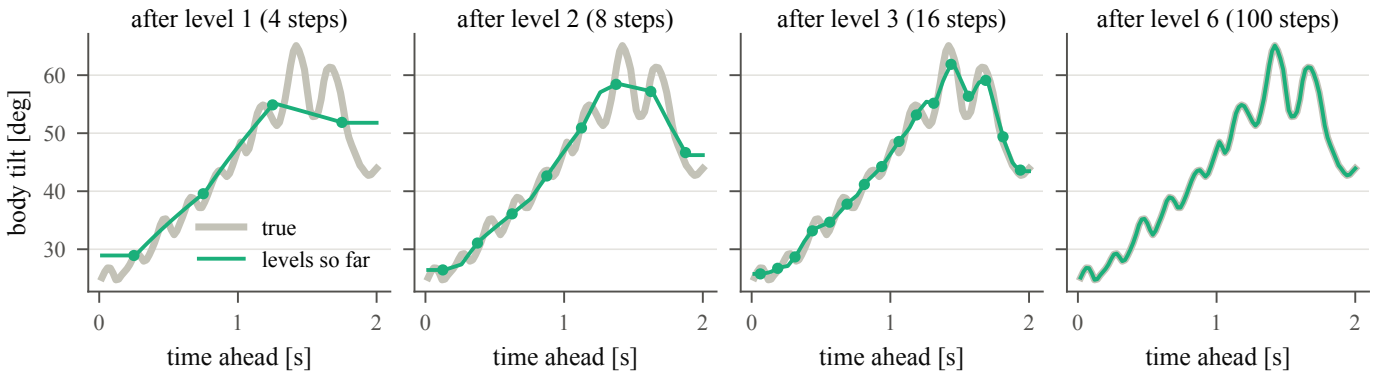


Fig. 2. How C-NCM builds a future. Body tilt over a 2 s window that ends in a fall (gray), and the sum of the levels generated so far (green). Level 1 has only four values but already shows the large rise in tilt. Later levels add the faster motion. After all six levels the trajectory is exact.

TABLE I
Main results on the 1,149 test anchors (319 forks). Ratio: mean predicted over mean true fall probability at forks, with 95% CI. Lower is better for Brier and energy. Bottom rows: after isotonic recalibration fit on a separate set of 1,163 anchors.

Model	Ratio at forks	Brier (forks)	Brier (all)	AUC	Energy
CFM	0.65 [0.57, 0.74]	0.0246	0.0196	0.899	0.373
Diffusion	0.57 [0.50, 0.65]	0.0254	0.0209	0.894	0.372
C-NCM (ours)	1.05 [0.95, 1.17]	0.0274	0.0209	0.902	0.431
Classifier	0.92 [0.82, 1.04]	0.0271	0.0218	0.881	–
CFM + isotonic	1.09	0.0245	0.0176	–	–
Diffusion + isotonic	1.12	0.0290	0.0183	–	–
C-NCM (ours) + isotonic	1.11	0.0248	0.0208	–	–

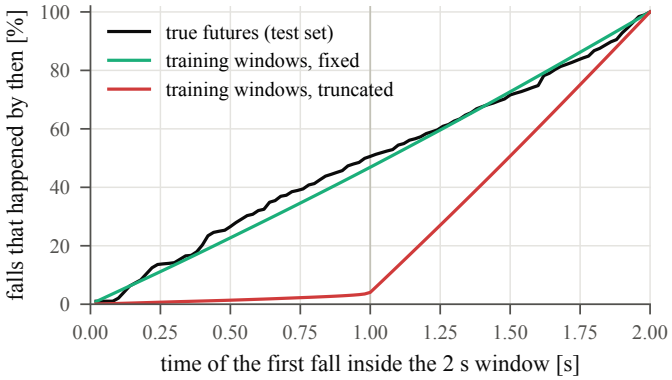


Fig. 3. The logging pitfall. For falls inside a 2 s window, the share that happened by each time. Real falls (black) are spread over the window, and half of them happen in the first second. When logging stops 1 s after a fall (red), windows that start just before a fall are never created, so models almost never see a fall in the first second. Logging 100 steps after a fall (green) fixes this.

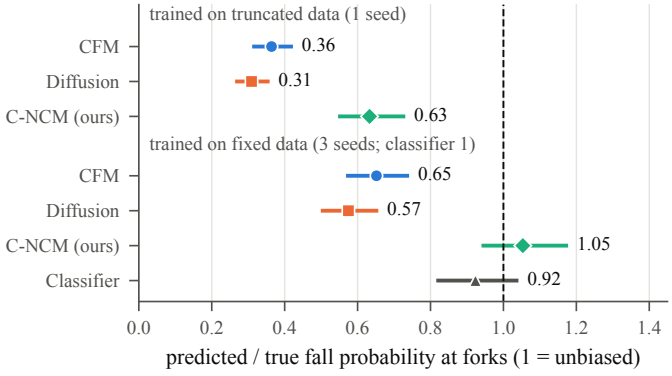


Fig. 4. Risk ratio at forks (mean predicted over mean true fall probability, 95% CI). Top: models trained on the truncated data. Bottom: the same models trained on the fixed data, and a classifier with the same inputs and network. Diffusion and CFM underestimate risk in both cases; C-NCM is unbiased on the fixed data.

episodes, where all 128 futures are identical, and many fall late in the window (after 1.5 s) on stairs or slopes. These are failures of the controller itself, decided by small details of the current state. They are a different problem from the random forks this paper is about, and no model we tried predicts them well.

F. Sampling steps

C-NCM uses more network passes than the flat models (384 against 64), and on the truncated data we had seen CFM's risk ratio grow with the number of steps. So we checked whether the flat models simply need more steps. We sampled the seed-0 models again with 16 to 512 steps, and C-NCM with 16 and 32 steps per level (96 and 192 passes), on the same test set (Fig. 8).

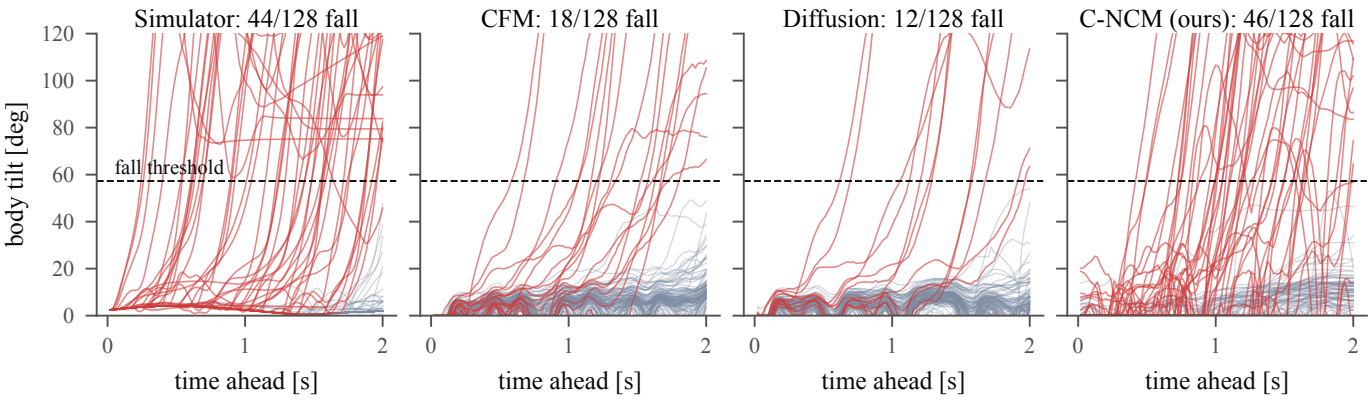


Fig. 5. One fork state (stairs, strong pushes). Body tilt of 128 futures from the simulator and 128 samples from each model (seed 0). Red lines end in a fall. The simulator gives 44 falls; CFM and diffusion give far fewer. C-NCM gives about the right number, but its samples are noisier in the first few steps, which is one reason for its worse energy distance.

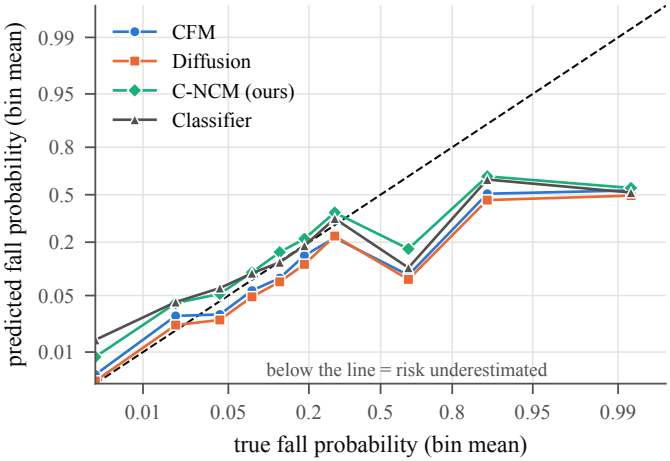


Fig. 6. Calibration by risk level. Anchors are grouped by true fall probability; each point is the mean prediction against the mean truth of a group (logit axes). Points below the dashed line underestimate risk.

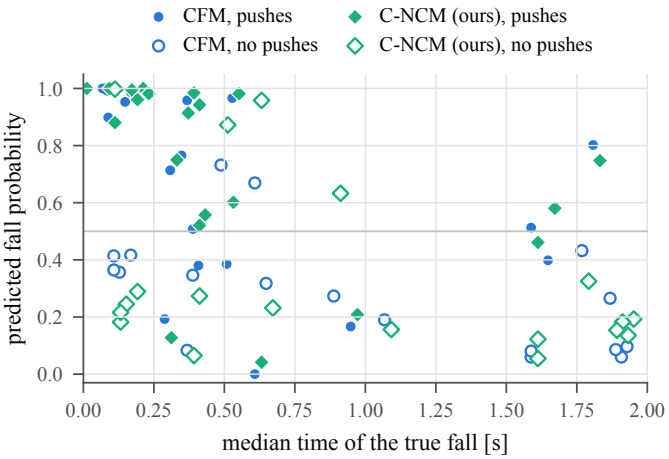


Fig. 7. States that are certain to fall ($p \geq 0.95$, 41 anchors): predicted probability against the median time of the true fall. Hollow markers are push-free episodes, where all futures are identical. Late, push-free falls are missed by every model.

More steps do help the flat models, but slowly. CFM goes from 0.67 at 64 steps to 0.75, 0.80 and 0.83 at 128, 256 and 512 steps; diffusion goes from 0.57 to 0.67, 0.73 and 0.77. Each doubling adds less than the one before, and even at 512 steps the upper end of the CI is below 1 (CFM 0.74–0.93, diffusion 0.68–0.86). C-NCM reaches 0.83 with 96 passes, 0.97 with 192 and 1.06 with 384. So, at an equal level of bias, C-NCM needs about five times fewer passes, and no flat model we ran reached an unbiased ratio. More steps did not change the per-state picture: CFM's fork Brier score stays at 0.0256–0.0257 from 128 to 512 steps.

This also tells us something about the bias itself. Few steps hurt the rare outcome most: at 16 steps CFM keeps only 29% of the true risk. The fall branch seems to be the part of the distribution that a coarse integration of the flat model loses first.

VII. Discussion

Why would flat models underestimate risk? We think the answer is in where the decision to fall is made. A fall is a coarse property of the whole two-second future. A flat diffusion or flow model makes that choice implicitly, while it denoises all 100×46 numbers at once, and a model trained with a squared loss tends to average over rare sub-modes [28]. C-NCM makes the choice first, on a 4-step summary where a fall and a recovery look very different, and only then fills in the detail. The step sweep fits this view: the flat models lose falls first when integration is coarse, and they recover them only slowly as steps are added. Our ablations on the truncated data agree with this picture: starting from a 4-step level worked better than starting from 16 steps, and removing the residuals or the corruption reduced the gain (Appendix B). This is a hypothesis that fits our results, not something we have proven.

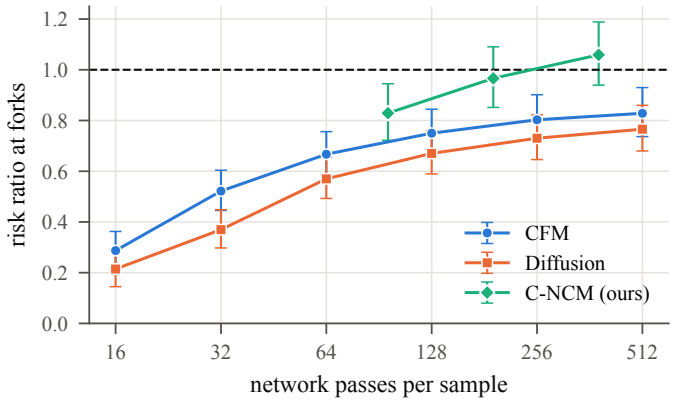


Fig. 8. Risk ratio against sampling compute (seed-0 models, test set, 95% CI). CFM and diffusion use one network pass per step; C-NCM uses one per step at each of its six levels. Flat models improve slowly with more steps and stay biased at 512 passes; C-NCM is unbiased from about 200 passes.

What does this mean in practice? If a team has held-out data with known outcomes, isotonic recalibration of a flat model's fall probability works as well as C-NCM for that number. Two things still favor a model that is calibrated by construction. First, many planners use the sampled futures directly, for example to average a cost over samples; a recalibrated probability does not change those samples. Second, recalibration needs outcome data from the same conditions, and a model that is biased in a predictable way can still be biased in an unpredictable way under new conditions. We did not test either point, and both deserve a direct test.

Limitations. All results are in simulation, with one robot, one hand-built controller and one kind of randomness (pushes). The controller has its own failures, which dominate the states that are certain to fall. We used one network size and one horizon (2 s). At the same number of steps per level, C-NCM is about 2.7 times slower to sample than CFM in our implementation (though at equal bias it is cheaper), and its trajectories are less accurate in detail. The step sweep used one seed. The ablations were run before the data fix and with one seed.

VIII. Conclusion

We measured something world models are rarely tested on: whether they predict failures as often as failures really happen. Exact branching in simulation gives the answer. Diffusion and flow-matching world models underestimate the risk of falling. Part of the gap came from a common logging pitfall, and the rest comes from the models themselves, since a classifier trained on the same data is nearly unbiased. Adding sampling steps narrows the gap only slowly. Generating the future coarse-to-fine with C-NCM removes the bias, although it does not improve how well states are ranked. We hope branching ground truth becomes a standard check for world models used in safety-critical planning.

References

[1] J. Ho, A. Jain, and P. Abbeel, "Denoising diffusion probabilistic models," in Advances in Neural Information Processing Systems, 2020.

[2] Y. Song, J. Sohl-Dickstein, D. P. Kingma, A. Kumar, S. Ermon, and B. Poole, "Score-based generative modeling through stochastic differential equations," in International Conference on Learning Representations, 2021.

[3] Y. Lipman, R. T. Q. Chen, H. Ben-Hamu, M. Nickel, and M. Le, "Flow matching for generative modeling," in International Conference on Learning Representations, 2023.

[4] X. Liu, C. Gong, and Q. Liu, "Flow straight and fast: Learning to generate and transfer data with rectified flow," in International Conference on Learning Representations, 2023.

[5] M. Janner, Y. Du, J. Tenenbaum, and S. Levine, "Planning with diffusion for flexible behavior synthesis," in International Conference on Machine Learning, 2022.

[6] E. Alonso, A. Jelley, V. Micheli, A. Kanervisto, A. Storkey, T. Pearce, and F. Fleuret, "Diffusion for world modeling: Visual details matter in Atari," in Advances in Neural Information Processing Systems, 2024.

[7] Z. Ding, A. Zhang, Y. Tian, and Q. Zheng, "Diffusion world model: Future modeling beyond step-by-step rollout for offline reinforcement learning," arXiv preprint arXiv:2402.03570, 2024.

[8] M. Rigter, J. Yamada, and I. Posner, "World models via policy-guided trajectory diffusion," Transactions on Machine Learning Research, 2024.

[9] D. Valevski, Y. Leviathan, M. Arar, and S. Fruchter, "Diffusion models are real-time game engines," in International Conference on Learning Representations, 2025.

[10] H. Kumbong, I. S. Khare, I. Mlauzi, A. Woen, C. Ding, E. K. Buchanan, S. Arora, A. Mirhoseini, and C. Ré, "Next-concept modeling: Fast, parallel coarse-to-fine text generation," arXiv preprint arXiv:2610.14162, 2026.

[11] K. Chua, R. Calandra, R. McAllister, and S. Levine, "Deep reinforcement learning in a handful of trials using probabilistic dynamics models," in Advances in Neural Information Processing Systems, 2018.

[12] D. Ha and J. Schmidhuber, "Recurrent world models facilitate policy evolution," in Advances in Neural Information Processing Systems, 2018.

[13] D. Hafner, T. Lillicrap, J. Ba, and M. Norouzi, "Dream to control: Learning behaviors by latent imagination," in International Conference on Learning Representations, 2020.

[14] D. Hafner, J. Pasukonis, J. Ba, and T. Lillicrap, "Mastering diverse control tasks through world models," Nature, vol. 640, 2025.

[15] P. Wu, A. Escontrela, D. Hafner, P. Abbeel, and K. Goldberg, "DayDreamer: World models for physical robot learning," in Conference on Robot Learning, 2022.

[16] C. Li, A. Krause, and M. Hutter, "Robotic world model: A neural network simulator for robust policy optimization in robotics," arXiv preprint arXiv:2501.10100, 2025.

[17] B. Chen, D. Martí Monsó, Y. Du, M. Simchowitz, R. Tedrake, and V. Sitzmann, "Diffusion forcing: Next-token prediction meets full-sequence diffusion," in Advances in Neural Information Processing Systems, 2024.

[18] Y. Yu, S. Zhang, Y. Sheng, H. Ren, and H. Lin, "How should world models be evaluated for embodied decision-making? A decision-making-centric position," arXiv preprint arXiv:2606.15032, 2026.

[19] Y. Li, Z. Zhou, Y. Chen, Y. Xue, and Y. Zhu, "dWorldEval: Scalable robotic policy evaluation via discrete diffusion world model," arXiv preprint arXiv:2604.22152, 2026.

[20] J. Liang, L. Jiang, K. Murphy, T. Yu, and A. Hauptmann, "The garden of forking paths: Towards multi-future trajectory prediction," in IEEE/CVF Conference on Computer Vision and Pattern Recognition, 2020.

[21] J. Lee, N. Merrill, and E. Karger, "ForecastBench-Sim: A simulated-world forecasting benchmark," arXiv preprint arXiv:2606.18686, 2026.

[22] Y. Ma, H. Shi, and X. Xu, "Twin rollouts: Noise-coupled counterfactual branching in interactive video world models," arXiv preprint arXiv:2608.08982, 2026.
[23] A. Q. Nichol and P. Dhariwal, "Improved denoising diffusion probabilistic models," in International Conference on Machine Learning, 2021.
[24] J. Song, C. Meng, and S. Ermon, "Denoising diffusion implicit models," in International Conference on Learning Representations, 2021.
[25] M. S. Albergo and E. Vanden-Eijnden, "Building normalizing flows with stochastic interpolants," in International Conference on Learning Representations, 2023.
[26] A. Tong, K. Fatras, N. Malkin, G. Huguet, Y. Zhang, J. Rector-Brooks, G. Wolf, and Y. Bengio, "Improving and generalizing flow-based generative models with minibatch optimal transport," Transactions on Machine Learning Research, 2024.
[27] R. Gao, E. Hoogeboom, J. Heek, V. De Bortoli, K. P. Murphy, and T. Salimans, "Diffusion meets flow matching: Two sides of the same coin," Blog post, https://diffusionflow.github.io/, 2024.
[28] Y. Lin, J. Shi, S. Ye, W. Wang, Y. Yao, and T. Liu, "SubFlow: Sub-mode conditioned flow matching for diverse one-step generation," arXiv preprint arXiv:2604.12273, 2026.
[29] Y. Huang, X. Hu, X. Lai, Z. Wang, L. Jian, J. Xu, and C. He, "Calibrating generative flows through regional probability matching," arXiv preprint arXiv:2610.16780, 2026.
[30] V. Sehwag, C. Hazirbas, A. Gordo, F. Ozgenel, and C. Canton Ferrer, "Generating high fidelity data from low-density regions using diffusion models," in IEEE/CVF Conference on Computer Vision and Pattern Recognition, 2022.
[31] S. Um, S. Lee, and J. C. Ye, "Don't play favorites: Minority guidance for diffusion models," in International Conference on Learning Representations, 2024.
[32] Y. C. Tsoi, K. M. R. Hunt, L. Shaffrey, A. Badii, R. Dixon, and L. Nicotina, "Using generative models to produce realistic populations of UK windstorms," arXiv preprint arXiv:2501.16110, 2025.
[33] S. K. Aithal, P. Maini, Z. C. Lipton, and J. Z. Kolter, "Understanding hallucinations in diffusion models through mode interpolation," in Advances in Neural Information Processing Systems, 2024.
[34] K. Tian, Y. Jiang, Z. Yuan, B. Peng, and L. Wang, "Visual autoregressive modeling: Scalable image generation via next-scale prediction," in Advances in Neural Information Processing Systems, 2024.
[35] J. Ho, C. Saharia, W. Chan, D. J. Fleet, M. Norouzi, and T. Salimans, "Cascaded diffusion models for high fidelity image generation," Journal of Machine Learning Research, vol. 23, 2022.
[36] O. Rippel, M. Gelbart, and R. Adams, "Learning ordered representations with nested dropout," in International Conference on Machine Learning, 2014.
[37] G. W. Brier, "Verification of forecasts expressed in terms of probability," Monthly Weather Review, vol. 78, no. 1, pp. 1–3, 1950.
[38] T. Gneiting and A. E. Raftery, "Strictly proper scoring rules, prediction, and estimation," Journal of the American Statistical Association, vol. 102, no. 477, pp. 359–378, 2007.
[39] C. Guo, G. Pleiss, Y. Sun, and K. Q. Weinberger, "On calibration of modern neural networks," in International Conference on Machine Learning, 2017.
[40] B. Zadrozny and C. Elkan, "Transforming classifier scores into accurate multiclass probability estimates," in ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, 2002.
[41] A. Niculescu-Mizil and R. Caruana, "Predicting good probabilities with supervised learning," in International Conference on Machine Learning, 2005.
[42] I. Price, A. Sanchez-Gonzalez, F. Alet, T. R. Andersson, A. El-Kadi, D. Masters, T. Ewalds, J. Stott, S. Mohamed, P. Battaglia, R. Lam, and M. Willson, "Probabilistic weather forecasting with machine learning," Nature, vol. 637, pp. 84–90, 2025.
[43] S. Kalyanakrishnan and A. Goswami, "Learning to predict humanoid fall," International Journal of Humanoid Robotics, vol. 8, no. 2, pp. 245–273, 2011.
[44] K. Srinivasan, B. Eysenbach, S. Ha, J. Tan, and C. Finn, "Learning to be safe: Deep RL with a safety critic," arXiv preprint arXiv:2010.14603, 2020.
[45] B. Thananjeyan, A. Balakrishna, S. Nair, M. Luo, K. Srinivasan, M. Hwang, J. E. Gonzalez, J. Ibarz, C. Finn, and K. Goldberg, "Recovery RL: Safe reinforcement learning with learned recovery zones," IEEE Robotics and Automation Letters, vol. 6, no. 3, 2021.
[46] J. Seo and A. Bajcsy, "Modeling latent disturbances for robust decision-making in world models," arXiv preprint arXiv:2610.07599, 2026.
[47] Z. Huang, Y. Cai, A. Patel, M. Hajiha, B. Browne, and Y. Chen, "Failure detection for surgical robot imitation policies via flow-matching world modeling," arXiv preprint arXiv:2607.27511, 2026.
[48] L. Buesing, T. Weber, Y. Zwols, S. Racaniere, A. Guez, J.-B. Lespiau, and N. Heess, "Woulda, coulda, shoulda: Counterfactually-guided policy search," in International Conference on Learning Representations, 2019.
[49] F. Pardo, A. Tavakoli, V. Levdik, and P. Kormushev, "Time limits in reinforcement learning," in International Conference on Machine Learning, 2018.
[50] J. Fu, A. Kumar, O. Nachum, G. Tucker, and S. Levine, "D4RL: Datasets for deep data-driven reinforcement learning," arXiv preprint arXiv:2004.07219, 2020.
[51] E. L. Kaplan and P. Meier, "Nonparametric estimation from incomplete observations," Journal of the American Statistical Association, vol. 53, no. 282, pp. 457–481, 1958.
[52] K. Zakka, Y. Tassa, and MuJoCo Menagerie Contributors, "MuJoCo Menagerie: A collection of high-quality simulation models for MuJoCo," https://github.com/google-deepmind/mujoco_menagerie, 2022.
[53] E. Todorov, T. Erez, and Y. Tassa, "MuJoCo: A physics engine for model-based control," in IEEE/RSJ International Conference on Intelligent Robots and Systems, 2012.
[54] M. H. Raibert, Legged Robots That Balance. MIT Press, 1986.
[55] G. J. Székely and M. L. Rizzo, "Energy statistics: A class of statistics based on distances," Journal of Statistical Planning and Inference, vol. 143, no. 8, pp. 1249–1272, 2013.
[56] B. Efron and R. J. Tibshirani, An Introduction to the Bootstrap. Chapman & Hall, 1994.
[57] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, "Attention is all you need," in Advances in Neural Information Processing Systems, 2017.
[58] I. Loshchilov and F. Hutter, "Decoupled weight decay regularization," in International Conference on Learning Representations, 2019.

Appendix

A. Pre-registration

The plan was written before any result of the final run. It fixed the training data (the regenerated 6,016 episodes), the test set (320 new episodes, never generated or looked at before), the calibration set, the three models and their settings (identical to the earlier runs), 64 sampling steps and 128 samples, the fork definition, averaging over three seeds, the bootstrap, the isotonic baseline and hypotheses H1–H4 with their decision rules. It also fixed the reading in advance: if H1 failed, the earlier underestimate would be reported as caused by the data pitfall, and a claim that C-NCM is better would need both H3 and H4. All secondary measures in this paper (all-anchor Brier, AUC, energy, calibration bins, the old models on the new test set) were listed there as "reported regardless of outcome". The classifier, the analysis of certain-fall states

and the sampling-step sweep were added after the test, as diagnostics.

B. Results on the truncated data and ablations

Before the data fix we ran a pre-registered test on another fresh set of 1,163 anchors (316 forks) with three seeds. Ratios were 0.35 (CFM, CI 0.31–0.39), 0.29 (diffusion, 0.25–0.32) and 0.59 (C-NCM, 0.52–0.67), and C-NCM had a lower fork Brier score than CFM (−0.0037, CI −0.0054 to −0.0021). Training CFM twice as long (80k steps) gave 0.34, and C-NCM without corruption gave 0.45. Table II lists the ablations, all on an exploratory set of 1,170 anchors (280 forks), one seed, 64 steps.

TABLE II
Ablations on the truncated data (exploratory set, seed 0, 64 steps).

Model	Ratio	Brier (forks)	Energy
CFM	0.35	0.0339	0.358
CFM, robust normalization	0.36	0.0338	0.359
Diffusion (DDIM)	0.29	0.0375	0.358
Diffusion (ancestral)	0.17	0.0436	0.388
C-NCM, levels 4–100 (main)	0.75	0.0262	0.410
without residuals	0.52	0.0329	0.416
two levels (4, 100)	0.56	0.0293	0.379
coarsest level 16 (16–100)	0.49	0.0301	0.393
full pyramid (1, 2, …, 100)	1.06	0.0357	0.430
Concept-first (k-means)	0.53	0.0511	0.519